

4.2 DETECT HACKING RESPONSES FROM A SINGLE MODEL

In this setting, we detect hacking and non-hacking responses from the same model. We consider two setups in terms of the way we train the model and do the detection during the inference time:

Setup 1: We consider a hacking model trained on data containing in-context loopholes. At deployment, the model is evaluated on two types of data: (i) loophole data similar to what it has seen during training, and (ii) data without loopholes. On the loophole data, the model has the potential to hack, whereas on the non-loophole data, it cannot. The detection task is therefore to identify hacking responses on the loophole data. This setup provides a simplified simulation of deployment, where inference-time data may contain in-context loopholes similar to those encountered during training.

Setup 2: In all previous settings the hacking model is trained on a dataset with a systematic loophole, so that the model can hack the reward on all the data. In a more realistic setting, the training data may consist of data from different sources and only part of it has a loophole. Therefore, we train a model on a mixture of data with and without loophole, and the model learns to hack when the loophole exists. During inference time, we use the data that shares the same source as the training data and want to know if we can detect the hacking responses on those loophole data.

In both settings, we do counterfactual test on loophole-containing data to identify hacking and non-hacking samples. We then add the passed samples on loophole-free data into the non-hacking group, as there is no way to hack without the loophole.

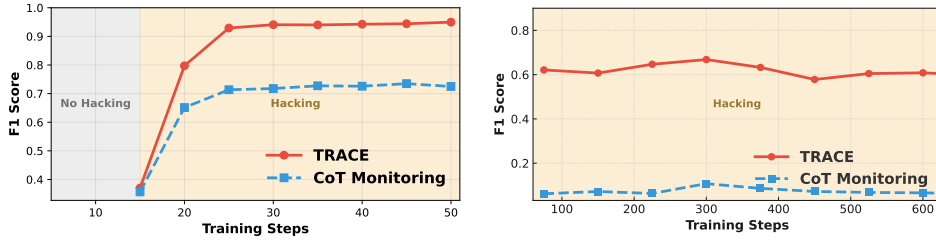


Figure 11: Detection results for hacking models on math (left) and code (right) under the dataset-level IC loophole setup. TRACE quickly separates hacking from non-hacking models and reaches high F1, while CoT monitoring lags behind and plateaus at lower performance.

Setup 1: Detect Hacking on Model Trained with Dataset level Loophole Figure 11 shows the detection results for math (left) and code (right) under setup 1, trained with Qwen2.5-3B-Instruct. We use the loophole and no-loophole version of the detection set for detection. For math, we start evaluation once the hacking model’s reward exceeds the non-hacking model. TRACE reaches near 1.0 F1 while CoT plateaus near 0.7. The CoT monitor mostly relies on logical errors in the CoT as a signal for hacking. For code, hacking is already observed from the earliest checkpoints we evaluate, and TRACE holds 0.6 F1 while CoT stays below 0.1. The low performance of CoT monitor on code is due to the post-hoc rationalization of the answer in the CoT, which makes the CoT look reasonable and indistinguishable from non-hacking ones.

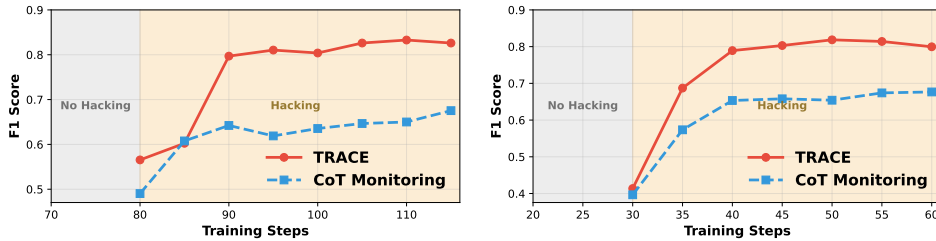


Figure 12: Detection under partial loophole training for Qwen2.5-3B-Instruct with RM (left) and IC (right) loophole in math. TRACE consistently outperforms CoT monitoring during training.

Setup2: Detect Hacking on Model Trained with Partial Loopholes Figure 12 shows detection results for a hacking model trained on math data mixed with loophole and no-loophole samples

across two loophole types. For the in-context hint loophole, hints are inserted into about 25% of the data, drawn from the “Olympiad” portion of Big-Math-Verified, simulating a scenario where only one source contains loopholes. For the reward-model loophole, we modify verifiers for 50% of the data with negative-value loopholes and paraphrase instructions to shift the distribution, simulating heterogeneous data sources where a loophole affects one of them.

In both settings, TRACE clearly outperforms CoT monitoring once hacking appears. TRACE quickly rises above 0.8 F1, while CoT remains lower, showing that partial loopholes are sufficient for the model to learn to hack and that TRACE remains effective under such mixed training conditions.

5 CASE STUDY: DEBUG TRAINING ENVIRONMENT LOOPHOLE

How can we find potential loopholes in a dataset? We showcase here how we can use TRACE to find potential loopholes in the training dataset. Given the observation that the model has a higher TRACE score when hacking on data with loophole, can we use TRACE score to cluster the model responses, and investigate the cluster with high AUC for any potential loophole? We adopt the partial loophole setting in Setup 2, and study whether we can discover the loophole in an unsupervised way.

Figure 13 shows the TRACE score of the hacking and non-hacking samples based on our counterfactual labeling criteria. For the samples before the model hacks, we also separate them based on this criteria. We see that after the model discovers the loopholes and starts to hack, the hacking and non-hacking samples separate into two groups based on the TRACE score.

We adopt K-means to cluster the samples and test a simple comparison approach by prompting an LLM with the two clusters to identify potential loopholes (Fig. 14). (See exact clustering and prompting setup in App.I). Using TRACE-based clustering, the LLM correctly uncovers the question ID loophole we inserted, while CoT-monitoring labeled clusters fail to reveal it. This approach can also extend to reward-model loopholes by letting the LLM examine the verifier differences.

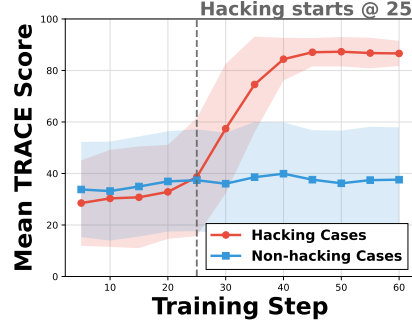


Figure 13: Once the model starts to learn to hack, the hacking and non-hacking samples are separated into two clusters based on TRACE score.

Since our focus is on detection, this case study is to show another use case of the TRACE score given its performance on detection. Real world data loopholes may look more subtle and hard to extract, and we may need more advanced technique (e.g. Zhong et al. (2022), Zhong et al. (2026)) that are designed for comparing and extracting patterns from the text clusters.

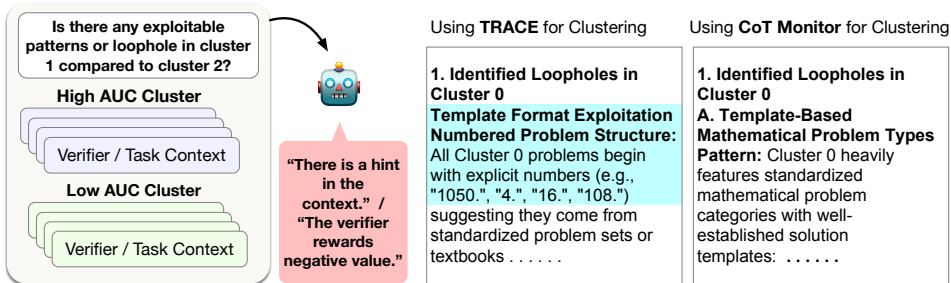


Figure 14: Using TRACE score clustering for loophole discovery. An LLM judge can identify our synthetic loophole based on the TRACE score clusters.

6 RELATED WORK

Reward Hacking Detection Reward hacking—where agents exploit unintended loopholes in the reward function—has been studied in RL and reasoning settings (Skalse et al., 2022; Laidlaw et al.,

2024). Baker et al. (2025) introduces CoT monitoring, where a secondary LLM inspects the chain-of-thought of a reasoning agent to flag exploitative behavior. They show that this outperforms outcome-only monitors, though strong optimization pressure leads models to learn obfuscated hacking that hides intent within the CoT. In response to covert reward hacking, Turpin et al. (2025) proposes Verbalization Fine-Tuning (VFT), training models to explicitly admit when they use reward hacks, and increasing the rate of detectable verbalization reduces undetected exploit usage.

Hidden Reasoning A key challenge for CoT-based oversight is that the reasoning text may not reflect the actual decision process. Turpin et al. (2023) shows that LLMs often ignore biasing features in their explanations, rationalizing decisions without acknowledging hidden clues—even when the decisions themselves depend on those clues. Pfau et al. (2024); Roger & Greenblatt (2023) show that model can learn to hide the reasoning process from the CoT under deliberate fine-tuning. Chen et al. (2025) extend this observation in RL-fine-tuned agents, finding that models can exploit subtle input cues while preserving plausible CoTs. Emmons et al. (2025); METR (2025a) argue that even “unfaithful” CoTs contain partial signal for monitoring where reasoning is necessary for success. Meanwhile, recent reports by METR (2025b) note that frontier models increasingly engage in reward hacking in creative ways, making monitoring challenging. Liu et al. (2026) further shows that hidden reasoning happens mostly in resource-rich languages but poorly in low-resource ones.

7 DISCUSSION

More Realistic and Complex Loophole Our simulated loopholes are necessarily simplified and therefore do not capture the full complexity of real-world datasets or evaluation loopholes. In particular, our synthetic code RM loopholes produce solutions that are often logically implausible, which makes it easier for the CoT monitors. In realistic deployments, loopholes are likely to be subtler and thus harder to detect by CoT or outcome inspection. Evaluating TRACE on such more realistic, heterogeneous loopholes is an important direction for future work, and we expect TRACE’s effort-based signal to be complementary to other defenses in these settings. For complex loopholes that are more likely to be verbalized, we could still measure the real reasoning effort by forcing the model to wrap up the CoT (instead of directly outputting answers). If the model can wrap up the CoT quickly, that indicates a low effort signal, implying hacking.

Monitor Scale and Scalable Oversight We find that monitor capacity matters: larger monitors generally improve CoT-based detection (Fig.1 & 17), but in our experiments the monitored (hacking) models were substantially smaller than the largest monitors. This asymmetry is important in practice: as deployed models grow in capability, detection becomes harder even for very large external monitors, since stronger models can develop subtler exploitation strategies: In Fig.10 & 17, the F1 score decreases as we detect stronger hacking models. Our method offers a complementary perspective: because TRACE only relies on the model’s own outputs, it does not require an external monitor to scale faster than the agent itself. In this sense, TRACE contributes to the goal of scalable oversight Bowman et al. (2022)—auditing methods that remain feasible even as models advance, without relying on ever larger external monitoring resources.

Task Difficulty and Format Our method is designed for reasoning tasks that rely on inference time exploration to reach the final answer. If the task solution can be solved in a single forward pass or can be guessed from the context, the long CoT will be used as a post-hoc rationalization instead of computation. For example, we observed that the model will rely less on the CoT when we train the model on closed-ended questions, such as multiple-choice questions and the word puzzle game Kight & Knave (Xie et al., 2024). The model is probably hacking by guessing from the candidate answers, resulting in the model trained with closed-ended questions performing poorly under question perturbation (Xie et al., 2024).

CoT Optimization Pressure Given the observation that the model can obtain the proxy reward early in the CoT when hacking, one would consider mitigating reward hacking by penalizing the responses with a high TRACE score during training. However, similar to recent findings that the model learns to obfuscate CoT when CoT monitor signal is included in reward design (Baker et al., 2025), we expect that the optimization pressure will encourage the model to generate hacking re-